



ENSIM
École d'ingénieurs
Le Mans Université



Projet Systèmes Embarqués : Système d'Éclairage Automatique Intelligent (STM32)

ESSOMBA MICHAEL, ARSÈNE NGUENANG YOUKAP

Année universitaire : 2025/2026

Tuteur université : KOSAI RAOOF

Table des matières

1	Introduction générale	3
2	Environnement de développement et méthodologie	3
2.1	Approche par validation incrémentale	4
3	Architecture matérielle	6
3.1	Vue d'ensemble du montage	6
3.2	Carte utilisée : NUCLEO-L496ZG	6
3.3	Composants externes	8
3.4	Répartition des broches et modes	9
3.5	Capteur de Présence PIR	10
3.6	Montage du capteur de luminosité (LDR)	10
3.7	Pilotage PWM du servomoteur	11
4	Principe de fonctionnement et logique décisionnelle	12
4.1	Objectif fonctionnel	12
4.2	Expression logique	12
4.3	Signalisation	12
5	Stratégie de validation et organisation des programmes	13
5.1	Programme 1 : validation du capteur de présence (PIR)	13
5.2	Programme 2 : validation du capteur de luminosité (LDR + ADC)	13
5.3	Programme 3 : validation du pilotage PWM du servomoteur	13
6	Conception logicielle (structure et extraits commentés)	14
6.1	Organisation générale d'un programme STM32CubeIDE	14
6.2	Gestion du capteur de présence (PIR)	14
6.2.1	Gestion d'événements : détection sur front	14
6.3	Lecture du capteur de luminosité (ADC)	15
6.4	Pilotage PWM (servo) : principe	15
6.5	Choix des paramètres PWM et conversion angle $\rightarrow$ impulsion	16
7	Résultats expérimentaux et validation	17
7.1	Validation du PIR	17
7.2	Affichage des messages sur le terminal série	17
7.3	Validation du LDR	18
7.4	Affichage des messages sur le terminal série	18
7.5	Validation du PWM	18

7.6	Cohérence globale du système	19
8	Analyse technique et limites	19
8.1	Délais et exécution bloquante	19
8.2	Robustesse des mesures ADC	19
8.3	Fiabilité de la détection PIR	19
9	Conclusion générale	20

1 Introduction générale

Dans un contexte de transition énergétique et d'optimisation des ressources, les systèmes d'éclairage intelligents représentent une solution efficace pour réduire la consommation électrique des bâtiments et infrastructures publiques. L'éclairage inutile, notamment dans les zones à occupation intermittente, constitue une source importante de gaspillage énergétique.

L'objectif de ce projet est de concevoir un système embarqué autonome capable d'activer un éclairage uniquement lorsque deux conditions sont simultanément vérifiées :

- la présence d'un individu détectée par un capteur PIR,
- un niveau de luminosité ambiante insuffisant mesuré via une LDR.

Le système repose sur une carte STM32 NUCLEO-L496ZG exploitant plusieurs périphériques internes : GPIO, ADC, Timer PWM et UART.

En raison de difficultés rencontrées lors de la fusion des différentes fonctionnalités (conflits de configuration CubeMX, erreurs de continuité logique et instabilités liées aux temporisations), le projet a été volontairement structuré en trois applications distinctes :

1. validation indépendante du capteur de présence,
2. validation indépendante du capteur de luminosité,
3. validation indépendante du pilotage PWM du servomoteur.

Cette approche incrémentale a permis d'isoler les erreurs, de stabiliser chaque sous-système, puis de vérifier la cohérence globale du fonctionnement.

Le présent rapport décrit l'architecture matérielle, la configuration logicielle et les choix techniques ayant conduit à un système fonctionnel et cohérent.

2 Environnement de développement et méthodologie

Le développement a été réalisé avec **STM32CubeIDE**, l'environnement officiel de STMicroelectronics. Cet outil intègre :

- **STM32CubeMX** pour la configuration graphique du microcontrôleur (affectation des broches, activation des périphériques, réglages des horloges),
- la génération automatique du code d'initialisation HAL (`MX_GPIO_Init`, `MX_ADC1_Init`, `MX_TIM3_Init`, etc.),
- la compilation (ARM GCC) et le débogage (via ST-LINK).

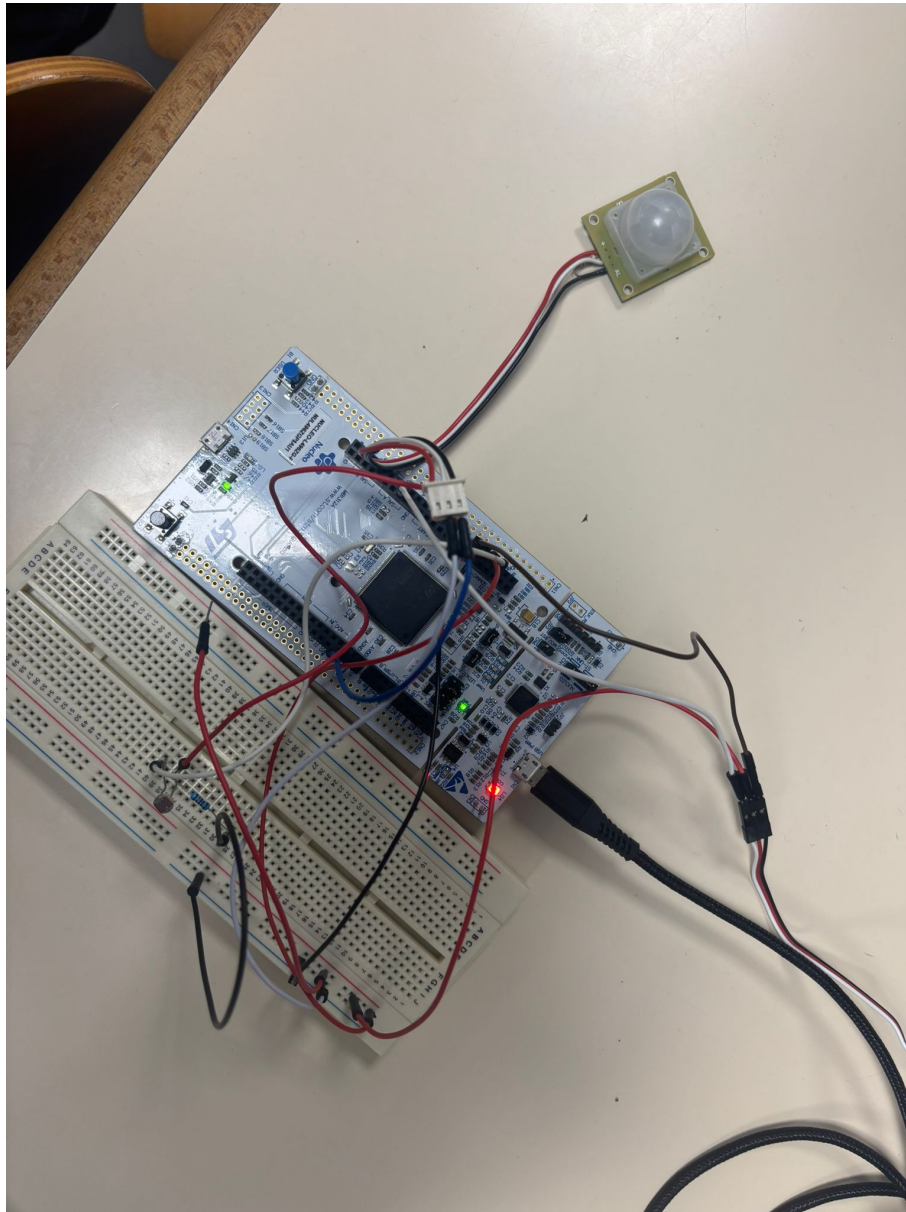


FIGURE 1 –

2.1 Approche par validation incrémentale

Afin de limiter les erreurs d'intégration et d'identifier rapidement les causes en cas de dysfonctionnement, le projet a été mené selon une approche incrémentale :

2 Environnement de développement et méthodologie

1. Configuration et test du PIR : lecture d'une entrée numérique, mise à jour d'une LED, envoi de messages sur terminal.
2. Configuration et test du LDR : conversion ADC, interprétation des valeurs, définition d'un seuil.
3. Configuration et test du PWM servo : génération d'un PWM stable et contrôle de positions.
4. Intégration finale : activation de l'éclairage simulé uniquement lorsque **présence ET luminosité faible**.

Cette méthode est particulièrement adaptée en systèmes embarqués, où les erreurs proviennent souvent d'un mélange entre problèmes matériels (câblage, niveaux logiques, alimentation) et problèmes logiciels (mauvaises broches, mauvaise configuration timer/ADC, délais bloquants).

3 Architecture matérielle

3.1 Vue d'ensemble du montage

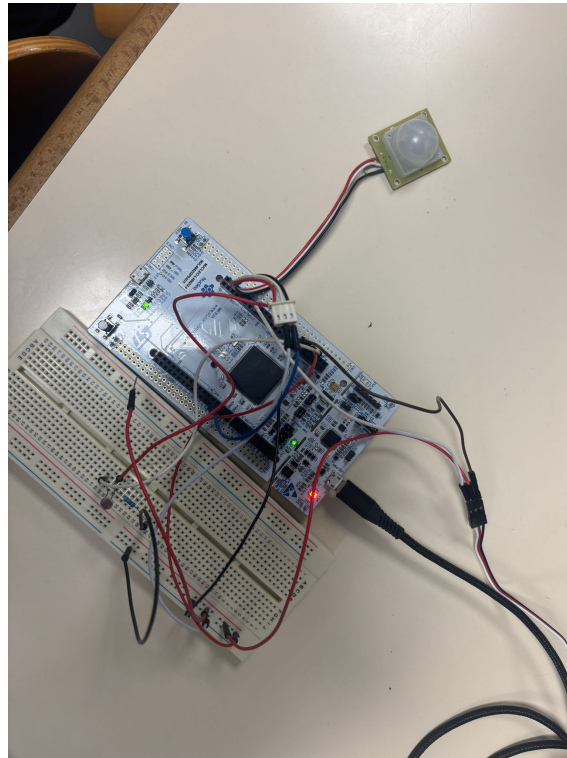


FIGURE 2 – Vue globale du montage (NUCLEO-L496ZG, LDR sur breadboard, PIR et servomoteur).

3.2 Carte utilisée : NUCLEO-L496ZG

La carte NUCLEO-L496ZG embarque un STM32L496ZG (Cortex-M4) et fournit :

- un accès direct aux GPIO via connecteurs,
- un ST-LINK intégré (programmation, debug),
- une alimentation simple via USB,
- des ressources internes adaptées : ADC 12 bits, timers PWM, interfaces série.

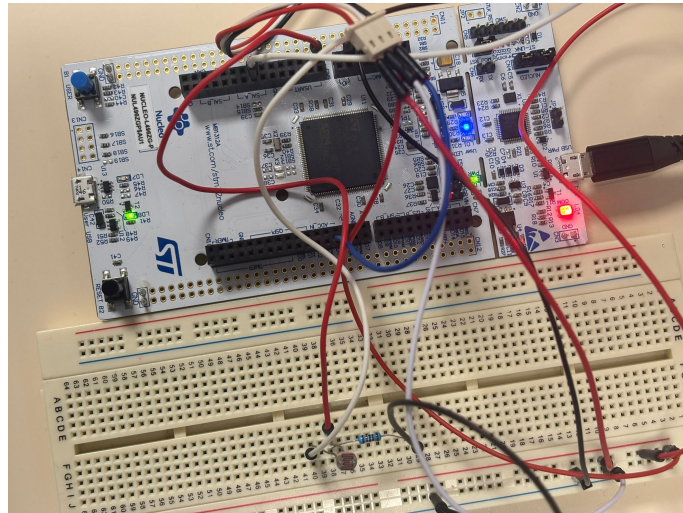


FIGURE 3 – Carte STM32 NUCLEO-L496ZG : vue rapprochée et câblage des périphériques.

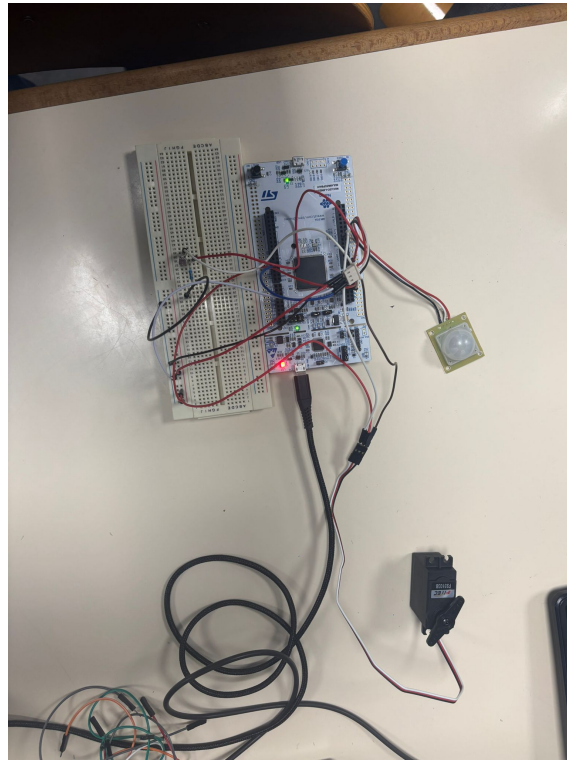


FIGURE 4 – Deuxième vue du montage : disposition des capteurs et de l'actionneur lors des essais.

3.3 Composants externes

Le système repose sur les composants suivants :

- **Capteur PIR** : sortie numérique indiquant présence/absence,
- **Capteur LDR** (avec résistance fixe) : montage en pont diviseur pour fournir une tension analogique,
- **Servomoteur FS5103B** : actionneur commandé en PWM,
- **LED verte** : indicateur présence,
- **LED rouge** : indicateur “éclairage requis / faible luminosité”,
- **Terminal série (TeraTerm)** : suivi des événements “Presence detectee” / “Absence detectee”.

3.4 Répartition des broches et modes

La configuration des broches correspond aux fonctions testées dans les différents programmes. La logique du rapport se base sur cette affectation (capteurs + indicateurs + actionneur).

Élément	Broche STM32	Configuration
Capteur PIR (sortie)	PF14	Entrée numérique + pull-down
LED verte (présence)	PB7	Sortie numérique (push-pull)
Capteur LDR (pont diviseur)	PA3 (ADC1_IN8)	Entrée analogique (ADC 12 bits)
LED rouge (luminosité faible)	PB14	Sortie numérique (push-pull)
Servomoteur FS5103B (PWM)	TIM3_CH2	Sortie PWM (Timer 3)
UART (messages terminal)	LPUART1	Transmission série (affichage)

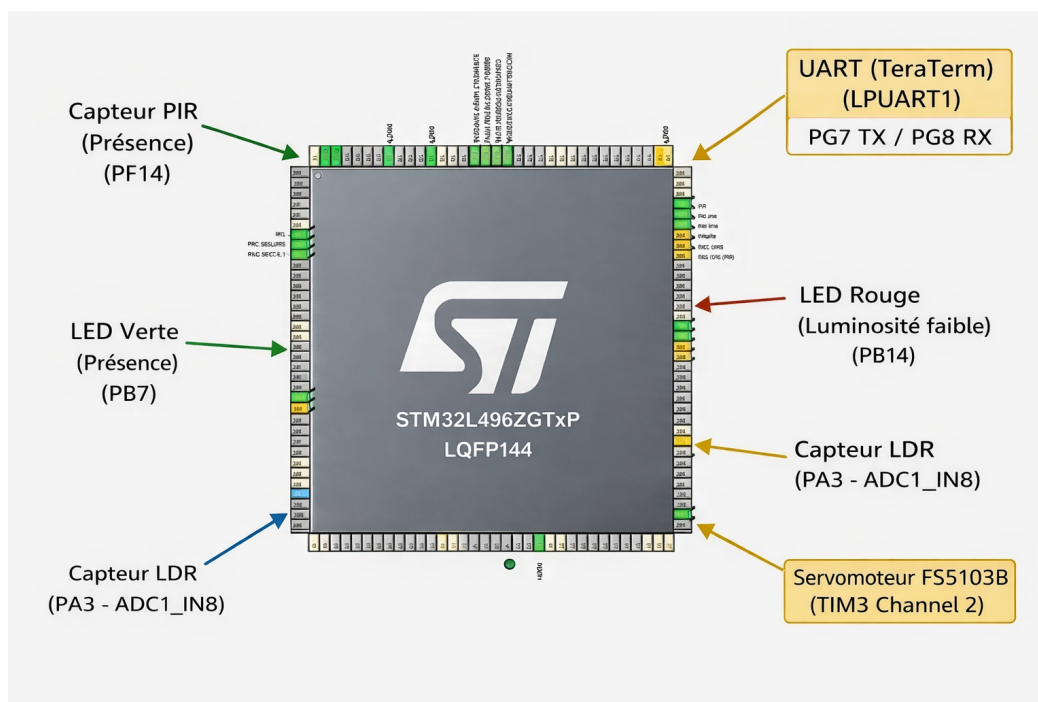


FIGURE 5 – Affectation des broches STM32 (PIR, LDR/ADC, LEDs, PWM servo, UART) telle que configurée dans STM32CubeIDE.

3.5 Capteur de Présence PIR

Le capteur PIR détecte les variations de rayonnement infrarouge émises par le corps humain. Caractéristiques :

- Sortie numérique
- Niveau logique haut : présence détectée
- Niveau logique bas : absence

Il est connecté à la broche PF14 configurée en entrée avec résistance pull-down.



FIGURE 6 – Capteur PIR utilisé pour la détection de présence.

3.6 Montage du capteur de luminosité (LDR)

La LDR est intégrée dans un **pont diviseur de tension** afin de transformer la variation de résistance en variation de tension mesurable par l'ADC :

$$V_{out} = V_{cc} \times \frac{R_{fixe}}{R_{fixe} + R_{LDR}}$$

Selon le câblage (position de la LDR dans le pont), une forte luminosité peut augmenter ou diminuer la tension. Le seuil logiciel est choisi en conséquence après observation des valeurs ADC.

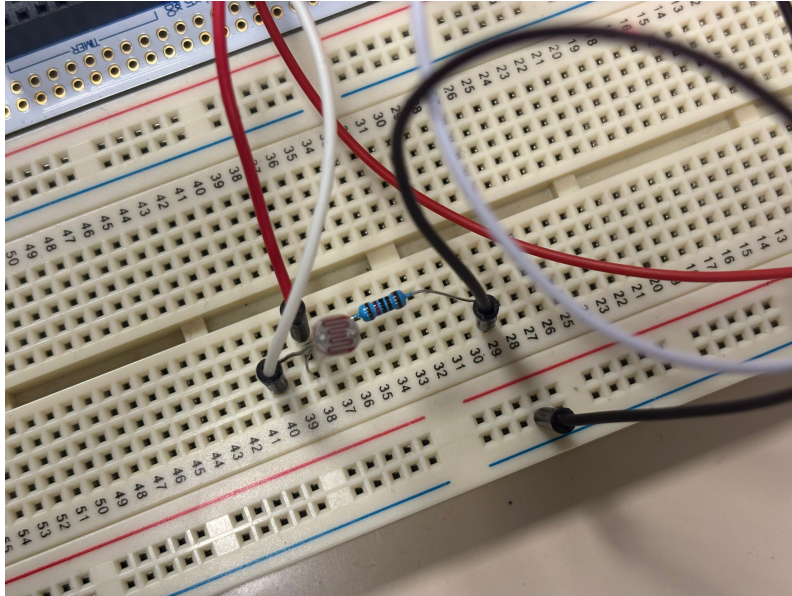


FIGURE 7 – Capteur LDR monté en pont diviseur (entrée analogique vers ADC1_IN8).

3.7 Pilotage PWM du servomoteur

Le servomoteur est commandé via un signal PWM. Dans notre configuration, un Timer du STM32 est configuré en PWM (mode PWM1) et la position est déterminée par la valeur de comparaison (CCR) :

- **Fréquence cible** : 50 Hz (période 20 ms),
- **Impulsion** : environ 1 ms à 2 ms selon l'angle (dépend du servo),
- l'ajustement de CCR modifie le rapport cyclique et donc l'angle.



FIGURE 8 – Servomoteur FS5103B utilisé comme actionneur (commande PWM).

4 Principe de fonctionnement et logique décisionnelle

4.1 Objectif fonctionnel

Le système doit activer l'éclairage uniquement si deux conditions sont réunies :

1. présence détectée (**PIR** = 1),
2. luminosité insuffisante (**ADC** < **seuil**).

4.2 Expression logique

$$Eclairage_ON = Presence \wedge (Luminosite < Seuil)$$

Cette approche est plus efficace qu'une activation basée uniquement sur la présence, car elle évite d'éclairer inutilement en plein jour ou dans un environnement déjà lumineux.

4.3 Signalisation

La supervision de l'état du système est assurée par :

- **LED verte** : reflète directement l'état du PIR (présence),
- **LED rouge** : indique l'état "éclairage requis" (faible luminosité),

- **terminal série** : messages alternés “Presence detectee” et “Absence detectee” selon les transitions.

5 Stratégie de validation et organisation des programmes

La tentative initiale de fusion des trois fonctionnalités au sein d'un seul projet a mis en évidence plusieurs problèmes : conflits de timers, incohérences dans la gestion des délais bloquants (HAL_Delay), *et difficultés de synchronisation entre la lecture ADC et la détection d'un événement*.

La séparation en trois programmes distincts a donc été un choix méthodologique visant à garantir la stabilité et la traçabilité des tests.

Afin de garantir un développement fiable, le projet a été validé de manière incrémentale en séparant les fonctionnalités en trois applications STM32 distinctes (trois fichiers `main.c`). Chaque application isole une fonction critique (présence, luminosité, actionneur) afin de faciliter le diagnostic matériel (câblage, niveaux logiques, alimentation) et logiciel (configuration CubeMX, périphériques internes).

5.1 Programme 1 : validation du capteur de présence (PIR)

Ce programme vérifie :

- la lecture stable de l'entrée PF14 (entrée numérique + pull-down),
- la cohérence des messages UART (TeraTerm),
- la logique événementielle (détection sur front : arrivée / départ).

5.2 Programme 2 : validation du capteur de luminosité (LDR + ADC)

Ce programme vérifie :

- la configuration de l'ADC1 sur le canal ADC1_IN8 (PA3),
- la lecture et l'interprétation des valeurs (0 à 4095 en 12 bits),
- l'utilisation d'un seuil de décision (ex : 3500) et la signalisation par LED.

5.3 Programme 3 : validation du pilotage PWM du servomoteur

Ce programme vérifie :

- la génération d'un PWM matériel stable via un timer,

- la correspondance entre la valeur de comparaison (CCR) et la position du servomoteur,
- la reproductibilité du mouvement (positions successives, vitesse, stabilité).

Cette organisation permet d'obtenir une validation robuste avant intégration complète du système.

6 Conception logicielle (structure et extraits commentés)

6.1 Organisation générale d'un programme STM32CubeIDE

La structure générée par CubeIDE repose sur deux zones principales :

- **Initialisation** (HAL + périphériques) : `HAL_Init()`, `SystemClock_Config()`, `MX_...Init()`,
- **Boucle principale** : `while(1)` contenant la logique temps réel.

6.2 Gestion du capteur de présence (PIR)

Le PIR est lu via une entrée numérique (PF14). L'extrait ci-dessous illustre une lecture simple, suivie de la mise à jour de la LED verte :

```
uint8_t raw = (HAL_GPIO_ReadPin(GPIOF, GPIO_PIN_14) == GPIO_PIN_SET);  
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, raw ? GPIO_PIN_SET : GPIO_PIN_RESET);
```

6.2.1 Gestion d'événements : détection sur front

Un point important dans un système de détection est d'éviter de répéter une action tant que le capteur reste à 1. On utilise donc une comparaison entre l'état courant et l'état précédent (détection de front montant / front descendant) :

```
static uint8_t last_raw = 0;  
  
if (raw == 1 && last_raw == 0)  
{  
    printf("Presence detectee\r\n");  
}  
  
if (raw == 0 && last_raw == 1)
```

```
{  
    UART_Print("Absence detectee\r\n");  
}  
  
last_raw = raw;
```

Cette méthode :

- déclenche un événement **une seule fois** à l'arrivée de présence,
- déclenche un événement **une seule fois** au départ de présence,
- évite les répétitions et facilite le diagnostic sur terminal.

6.3 Lecture du capteur de luminosité (ADC)

La LDR fournit une tension analogique convertie par l'ADC1 (résolution 12 bits).
La séquence HAL classique est :

```
HAL_ADC_Start(&hadc1);  
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);  
uint32_t valeur_adc = HAL_ADC_GetValue(&hadc1);
```

La valeur `valeur_adc` est ensuite comparée à un **seuil** déterminé expérimentalement. Le seuil dépend :

- du montage du pont diviseur (position LDR / résistance fixe),
- de l'éclairage ambiant réel,
- de la tension d'alimentation et du bruit sur la mesure.

6.4 Pilotage PWM (servo) : principe

Le pilotage du servo est basé sur la mise à jour du registre CCR via :

```
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, valeur_ccr);
```

Dans les tests, une alternance de positions permet de visualiser immédiatement le mouvement et de confirmer la stabilité du PWM.

6.5 Choix des paramètres PWM et conversion angle $\rightarrow$ impulsion

Le servomoteur FS5103B est commandé par un signal PWM de fréquence typique 50 Hz (période 20 ms). L'information de position est portée par la largeur d'impulsion, généralement comprise entre environ 1 ms et 2 ms selon le modèle du servomoteur.

Dans le programme de validation du servomoteur, un timer est configuré de manière à obtenir une période compatible avec 20 ms. La valeur **ARR** (Period) fixe la période de comptage, et **CCR** (Pulse) fixe la durée à l'état haut.

La fonction de commande implémente une conversion linéaire entre l'angle demandé et une valeur de comparaison :

```
pulse_lenght = 210 + (angle*(1050-210)/180);  
__HAL_TIM_SET_COMPARE(htim, channel, pulse_lenght);
```

Ainsi :

- **210** correspond à l'impulsion minimale (position proche de 0°),
- **1050** correspond à l'impulsion maximale (position proche de 180°),
- une interpolation linéaire assure un mouvement progressif et reproductible.

Lors des tests, la commande par paliers (ex : 0°, 30°, 60°...) permet de vérifier rapidement la stabilité du PWM, la réponse mécanique et l'absence de titter visible.

Justification du choix des paramètres Timer

Le timer est configuré avec un prescaler et une valeur de période permettant d'obtenir une fréquence proche de 50 Hz.

La fréquence PWM est donnée par :

$$f_{PWM} = \frac{f_{clock}}{(PSC + 1)(ARR + 1)}$$

où :

- PSC est le prescaler,
- ARR est la valeur d'auto-rechargement,
- f_{clock} est la fréquence du timer.

Le choix de ces paramètres garantit une période proche de 20 ms, adaptée aux servomoteurs standards.

Le mapping linéaire entre angle et largeur d'impulsion simplifie la commande tout en assurant :

- une précision suffisante,

- une stabilité mécanique,
- une absence de vibrations visibles.

7 Résultats expérimentaux et validation

7.1 Validation du PIR

Les tests montrent :

- une lecture stable de PF14 grâce au **pull-down** (évite l'état flottant),
- une synchronisation immédiate de la LED verte,
- des messages série cohérents, basés sur les transitions : `Presence detectee` puis `Absence detectee`.

7.2 Affichage des messages sur le terminal série

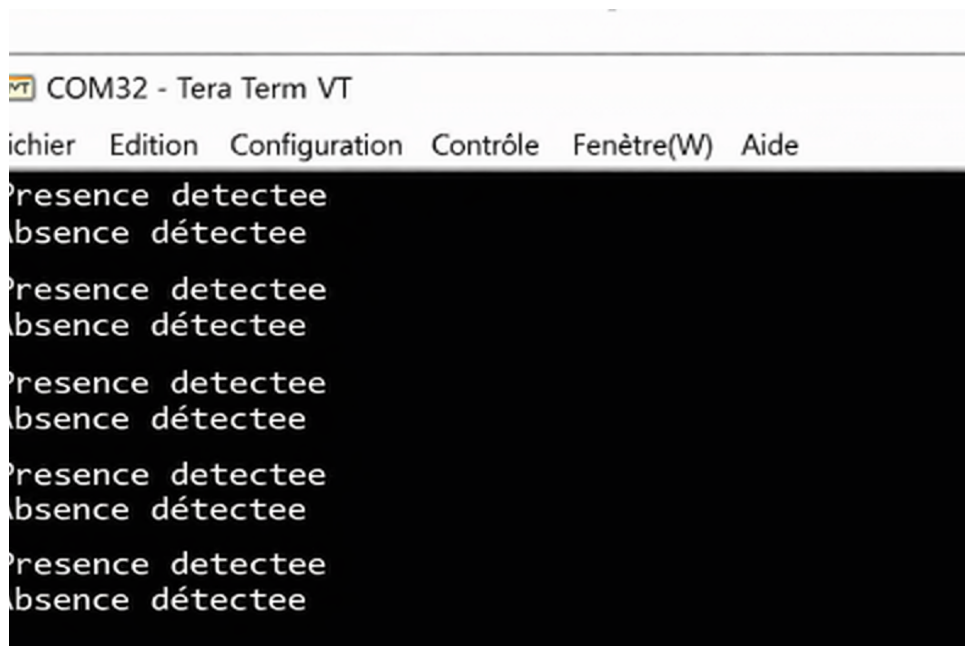


FIGURE 9 – Affichage TeraTerm : alternance "Presence detectee" / "Absence detectee".

7.3 Validation du LDR

La conversion ADC permet d'obtenir une mesure exploitable de la luminosité. Après choix d'un seuil :

- le système distingue clairement un environnement "lumineux" d'un environnement "sombre",
- la LED rouge permet de visualiser le résultat de la condition de luminosité.

7.4 Affichage des messages sur le terminal série

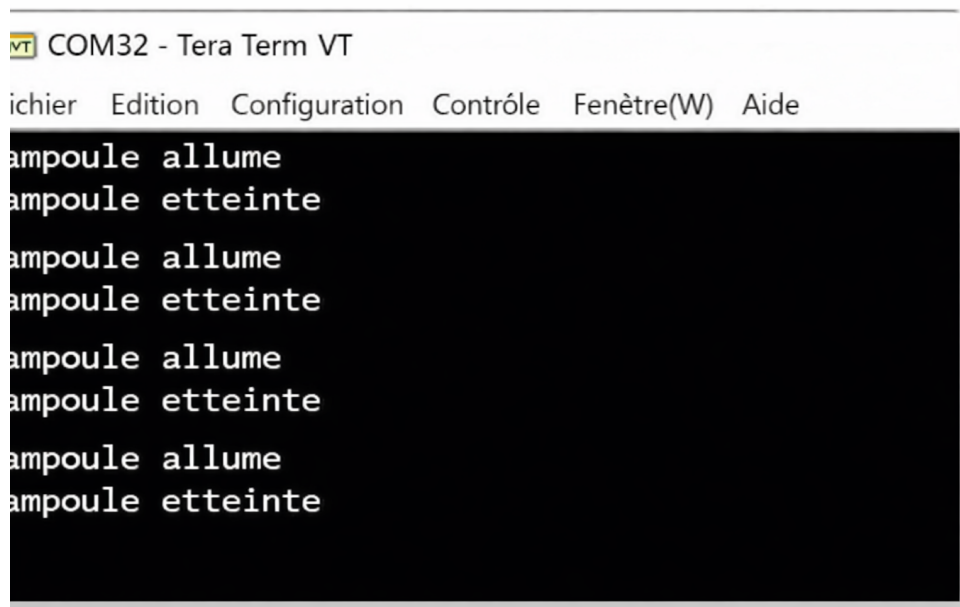


FIGURE 10 – Affichage TeraTerm : alternance "ampoule allume" / "ampoule etteinte".

7.5 Validation du PWM

Le servomoteur répond aux consignes de position imposées par la valeur CCR. L'utilisation du Timer 3 en PWM garantit :

- une génération stable et matérielle du signal (moins dépendante de la boucle logique),
- une commande reproductible des positions.

7.6 Cohérence globale du système

Bien que les fonctionnalités aient été validées séparément, l'analyse fonctionnelle confirme la cohérence du comportement global attendu :

$$Eclairage_ON = Presence \wedge (Luminosite < Seuil)$$

Les tests montrent que :

- la détection de présence est fiable et déclenche correctement les événements,
- la mesure de luminosité permet une discrimination claire entre environnement sombre et lumineux,
- le servomoteur réagit de manière stable aux commandes PWM.

La séparation des programmes a permis de valider chaque sous-système indépendamment, garantissant que les dysfonctionnements observés lors des tentatives de fusion provenaient principalement de problèmes de synchronisation logicielle et non d'erreurs matérielles.

8 Analyse technique et limites

8.1 Délais et exécution bloquante

Les appels à `HAL_Delay()` simplifient la validation, mais rendent la boucle principale bloquante. Pour une version industrielle, on privilégierait :

- une gestion par interruptions,
- ou une base de temps via `SysTick` et une machine d'états non bloquante.

8.2 Robustesse des mesures ADC

La mesure de luminosité peut être améliorée via :

- filtrage logiciel (moyenne glissante),
- hystérésis sur le seuil (évite les oscillations proche du seuil),
- suréchantillonnage si nécessaire.

8.3 Fiabilité de la détection PIR

Le PIR peut produire des impulsions parasites selon l'environnement (bruit, chaleur, mouvements non humains). Une validation plus poussée peut inclure :

- temporisation de validation de présence,
- maintien de l'éclairage actif pendant un temps minimal après détection.

9 Conclusion générale

Ce projet a permis de concevoir et de valider un système d'éclairage automatique intelligent basé sur une architecture STM32.

Au-delà de l'aspect fonctionnel, le travail a mis en évidence l'importance d'une méthodologie incrémentale en systèmes embarqués. La tentative de fusion des différentes fonctionnalités dans un seul projet a révélé des problématiques classiques : conflits de configuration des périphériques, gestion bloquante des délais et interactions imprévues entre modules.

La séparation en trois applications distinctes a constitué un choix stratégique permettant :

- d'isoler les erreurs,
- de stabiliser chaque sous-système,
- de garantir la validité des mesures expérimentales.

Le système final démontre la maîtrise des périphériques essentiels du STM32 :

- GPIO pour la détection et la signalisation,
- ADC pour l'acquisition analogique,
- Timer PWM pour le pilotage d'actionneur,
- UART pour la supervision et le diagnostic.

Ce projet illustre une démarche rigoureuse de développement embarqué, conciliant conception matérielle, configuration logicielle et validation expérimentale.